

Programming for Problem Solving (USA24102J)

Unit I

1.1.Evaluation of Programming Languages, Problem Solving through Programming

Evaluation of Programming Languages

Evaluation of programming languages is the process of assessing and comparing programming languages based on their features, performance, usability, and suitability for developing different types of software applications.

Evaluation Criteria

Criterion	Description
Readability	The ease with which programmers can read and understand the code.
Writability	The ease of writing programs using the language.
Reliability	The ability of a program to perform correctly under all conditions.
Efficiency	The effective use of CPU time, memory, and other system resources.
Portability	The ability of programs to run on different hardware and operating systems with minimal changes.
Maintainability	The ease of modifying, updating, and debugging programs.
Security	The capability of the language to prevent errors and unauthorized access.
Simplicity	The language has simple syntax and fewer complex rules, making it easier to learn and use.

Importance of Evaluating Programming Languages

- Helps select the most suitable language for a project.
- Improves software quality and reliability.
- Increases programmer productivity.
- Reduces development and maintenance costs.
- Ensures better performance and scalability.
- Supports portability across different platforms.

Common Programming Languages and Their Strengths

Language	Strength
C	Fast execution and system programming.
C++	Object-oriented programming and high performance.
Java	Platform-independent, secure, and robust.
Python	Simple syntax, rapid development, AI, and data science.
JavaScript	Interactive web application development.

Language	Strength
C	Fast execution and system programming.
C++	Object-oriented programming and high performance.
Java	Platform-independent, secure, and robust.
Python	Simple syntax, rapid development, AI, and data science.
JavaScript	Interactive web application development.

Example

Suppose a company wants to develop a mobile application:

- **Java/Kotlin** are preferred for Android development.
- **Swift** is preferred for iOS development.
- **Python** may be used for the application's backend because of its simplicity and rich libraries.

Advantages

- Helps choose the right programming language.
- Enhances software efficiency and reliability.
- Simplifies program development and maintenance.
- Supports reusable and scalable software solutions.

Disadvantages

- No single language is best for every application.
- Different languages have different learning curves.
- Performance and features vary depending on the application domain.

Problem Solving through Programming

Problem solving through programming is the process of:

1. **Understanding a problem** – Identify inputs, outputs, and constraints.
2. **Designing an algorithm** – Develop a step-by-step solution.
3. **Implementing the algorithm** – Write the solution in a programming language.
4. **Testing and debugging** – Verify correctness and fix errors.
5. **Optimizing** – Improve efficiency in terms of time and memory.

Example: Add Two Numbers

Problem: Write a program to add two numbers.

Algorithm:

1. Start.
2. Read two numbers, A and B.
3. Calculate $\text{Sum} = A + B$.
4. Display the sum.
5. Stop.

Python Program:

```
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))  
sum = a + b  
print("Sum =", sum)
```

Input:

Enter first number: 15

Enter second number: 25

Output:

Sum = 40

1.2. Writing Algorithms, Pseudo Code, Drawing flowchart

Algorithm

An **algorithm** is a finite, step-by-step procedure used to solve a problem or perform a specific task.

Algorithm:

1. Start.
2. Read two numbers A and B.
3. Compute $\text{Sum} = A + B$.
4. Display Sum.
5. Stop.

Example: Algorithm to Find the Largest of Two Numbers

Problem: Find the largest of two numbers.

Algorithm:

1. Start.
2. Read two numbers **A** and **B**.
3. If $A > B$, display **A**.
4. Otherwise, display **B**.
5. Stop.

Characteristics of a Good Algorithm

- **Input:** Accepts zero or more inputs.
- **Output:** Produces at least one output.
- **Definiteness:** Every step is clear and unambiguous.
- **Finiteness:** Ends after a finite number of steps.
- **Effectiveness:** Each step is simple and executable.

Pseudocode

Pseudocode is an informal, language-independent way of describing the steps of an algorithm using simple English and programming-like statements.

Example 1: Pseudocode to Add Two Numbers

BEGIN

 INPUT A, B

 SUM = A + B

 PRINT SUM

END

Example 2: Pseudocode to Find the Largest of Two Numbers

BEGIN

 INPUT A, B

 IF $A > B$ THEN

 PRINT A

 ELSE

 PRINT B

 END IF

END

Example 3: Pseudocode to Find the Sum of First N Natural Numbers

BEGIN

INPUT N

$SUM \leftarrow N \times (N + 1) / 2$

PRINT SUM

END

Advantages of Pseudocode

- Easy to read and understand.
- Independent of any programming language.
- Helps in designing algorithms before coding.
- Makes debugging and program development easier.
- Improves communication among programmers and users.

Drawing Flowcharts

Definition:

A **flowchart** is a graphical representation of an algorithm or process using standard symbols connected by arrows to show the sequence of steps.

Common Flowchart Symbols

Symbol	Purpose	
Oval	Start/End	
Parallelogram	Input/Output	
Rectangle	Process/Calculation	
Diamond	Decision (Yes/No)	
Arrow	Flow of control	

Example: Flowchart to Add Two Numbers

Algorithm: 1. Start. 2. Read two numbers A and B . 3. Sum = A + B . 4. Print Sum . 5. Stop.	Flowchart : <pre>graph TD; Start([Start]) --> Read[/Read A,B/]; Read --> Sum[Sum=A+B]; Sum --> Print[/Print Sum/]; Print --> Stop([Stop])</pre>
---	---

Steps to Draw a Flowchart

1. Identify the problem.
2. List the sequence of steps.
3. Use the appropriate flowchart symbols.
4. Connect the symbols with arrows.
5. Review the flowchart for correctness.

Advantages of Flowcharts

- Easy to understand and communicate.
- Helps visualize program logic.
- Simplifies debugging and testing.
- Serves as documentation.
- Makes program development more systematic.

1.3 Evaluation of C Language Program Structure

The structure of a C program refers to the standard organization of its components, which makes the program easy to read, write, debug, and maintain.

A typical C program structure includes:

1. **Documentation Section** – Contains comments that describe the program, author, and purpose.
2. **Link Section** – Includes header files using the `#include` directive.
3. **Definition Section** – Defines symbolic constants using `#define`.
4. **Global Declaration Section** – Declares global variables and function prototypes.

5. **main() Function** – The entry point of the program where execution begins.
6. **Declaration Part** – Declares local variables inside the main() function.
7. **Executable Part** – Contains statements, expressions, input/output operations, and control structures.
8. **User-Defined Functions** – Contains functions written by the programmer to perform specific tasks.

Example

```
/* Documentation Section */

#include <stdio.h>           // Link Section

#define PI 3.14              // Definition Section

int square(int);             // Global Declaration

int main()                   // Main Function
{
    int num, result;         // Declaration Part

    printf("Enter a number: ");
    scanf("%d", &num);

    result = square(num);    // Executable Part

    printf("Square = %d", result);

    return 0;
}

/* User-Defined Function */

int square(int n)
{
    return n * n;
}
```

1.7 Need for file header, Need for linkers and loaders

File Header

A **file header** is the beginning section of a source code file that contains information about the program.

Need for a File Header

- Identifies the **program name** and purpose.

- Provides **author** and version information.
- Records the **date of creation and modification**.
- Improves **code readability** and documentation.
- Helps in **maintenance** and future updates.
- Makes collaboration easier in team projects.

Example of a File Header in C

```
/*
Program Name : Student Management System
Author      : John Doe
Date       : 17-07-2026
Version    : 1.0
Description : Program to manage student records.
*/
```

Linkers

A **linker** is a system program that combines object files and libraries into a single executable program.

Need for Linkers

- Combines multiple object files into one executable.
- Resolves references to external functions and variables.
- Links standard and user-defined libraries.
- Detects unresolved symbols and linking errors.
- Produces the final executable file for execution.

Example:

If a program uses `printf()`, the linker connects the program with the C standard library where `printf()` is defined.

Loaders

A **loader** is a system program that loads the executable file into the computer's main memory for execution.

Need for Loaders

- Loads the executable program into RAM.
- Allocates memory for the program.
- Initializes the program before execution.

- Resolves address references (if required).
- Transfers control to the program's `main()` function to start execution.

Example:

When you run an executable the loader places it in memory and starts its execution.

Component	Purpose
File Header	Provides program documentation and identification .
Linker	Combines object files and libraries to create an executable program.
Loader	Loads the executable into memory and starts program execution .

1.8 Inputs and outputs statements : `scanf`, `printf`, Identifiers and variables

Input and output (I/O) statements are used to communicate between the user and the program. Input statements receive data from the user, while output statements display data on the screen.

`scanf()` Function in C

The **`scanf()`** function is a standard input function in C used to read formatted input from the keyboard. It is declared in the `<stdio.h>` header file.

Syntax

```
scanf("format_specifier", &variable);
```

Parameters

- **format_specifier**: Specifies the type of data to be read (e.g., `%d`, `%f`, `%c`).
- **&variable**: The address of the variable where the input value will be stored.

Example

```
#include <stdio.h>

int main()
{
    int age;
    printf("Enter your age: ");
    scanf("%d", &age);
    printf("Age = %d", age);
    return 0;
}
```

Output

Enter your age: 20

Age = 20

Common Format Specifiers

Format Specifier	Data Type
%d	Integer
%f	Float
%lf	Double
%c	Character
%s	String
%u	Unsigned Integer
%x	Hexadecimal Integer

Example: Reading Multiple Values

```
#include <stdio.h>

int main()
{
    int age;
    float salary;
    printf("Enter age and salary: ");
    scanf("%d %f", &age, &salary);
    printf("Age = %d\nSalary = %f", age, salary);
    return 0;
}
```

Identifiers (variables) in C

An **identifier** is a name given to program elements such as **variables, functions, arrays, structures, constants, and labels**. Identifiers help programmers refer to these elements in a program.

A **variable** is a named memory location used to store data. The value stored in a variable can **change during the execution** of a program.

Rules for Naming Identifiers

- Must begin with a **letter** (A–Z, a–z) or an **underscore** (_).
- Can contain **letters, digits (0–9), and underscores** (_).
- Cannot begin with a digit.
- Cannot contain spaces or special characters (such as @, #, %, &).
- Cannot be a **C keyword** (e.g., int, if, while).
- C is **case-sensitive**, so Total and total are different identifiers.

Valid Identifiers

age

studentName

_marks

total123

sum_of_numbers

Invalid Identifiers

123age // Starts with a digit

student name // Contains a space

total-amount // Contains a hyphen (-)

int // C keyword

@value // Contains a special character

Example Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int age = 20;
```

```
    float salary = 25000.50;
```

```
    printf("Age = %d\n", age);
```

```
    printf("Salary = %.2f", salary);
```

```
    return 0;
```

```
}
```

Constants in C

A **constant** is a fixed value that **cannot be changed** during the execution of a program. Unlike variables, the value of a constant remains the same throughout the program.

Types of Constants

1. Integer Constants

Whole numbers without a decimal point.

Examples:

10

-25

0

1000

2. Floating-Point Constants

Numbers with a decimal point or exponent.

Examples:

3.14

-12.5

0.001

2.5e3

3. Character Constants

A single character enclosed in single quotes.

Examples:

'A'

'5'

'#'

4. String Constants

A sequence of characters enclosed in double quotes.

Examples:

"Hello"

"C Programming"

"SRMIST"

5. Symbolic Constants

Constants defined using the #define preprocessor directive.

Syntax

```
#define CONSTANT_NAME value
```

Example

```
#define PI 3.14159
```

6. Constant Variables

Constants declared using the const keyword.

Syntax

```
const data_type variable_name = value;
```

Example

```
const int MAX = 100;
```

Example Program

```
#include <stdio.h>
```

```
#define PI 3.14159
```

```
int main()
```

```
{
```

```
    const int MAX = 100;
```

```
    printf("PI = %.5f\n", PI);
```

```
    printf("MAX = %d\n", MAX);
```

```
    return 0;
```

```
}
```

Output

```
PI = 3.14159
```

```
MAX = 100
```

1.9 Keywords, Literals

Keywords in C

Keywords are reserved words in the C programming language that have predefined meanings. They are used to perform specific tasks and **cannot be used as identifiers** (such as variable names, function names, or array names).

Features of Keywords

- Have predefined meanings in C.
- Cannot be used as variable or function names.
- Written in **lowercase**.
- Form the basic building blocks of C programs.
- Used to define data types, control statements, storage classes, and other language constructs.

Common C Keywords (32)

Keyword	Purpose
auto	Declares automatic variables
break	Terminates a loop or switch
case	Defines a case in a switch statement
char	Declares a character data type
const	Declares a constant variable
continue	Skips the current loop iteration
default	Default case in a switch statement
do	Starts a do-while loop
double	Declares a double-precision floating-point variable
else	Alternative branch in an if statement
enum	Declares an enumeration
extern	Declares an external variable
float	Declares a floating-point variable
for	Starts a for loop
goto	Transfers control to a labeled statement
if	Executes code based on a condition
int	Declares an integer variable
long	Declares a long integer
register	Declares a register variable

Keyword	Purpose
return	Returns control from a function
short	Declares a short integer
signed	Declares a signed data type
sizeof	Returns the size of a data type or variable
static	Declares a static variable
struct	Declares a structure
switch	Performs multiple selections
typedef	Creates a new name for a data type
union	Declares a union
unsigned	Declares an unsigned data type
void	Indicates no value or return type
volatile	Indicates a variable may change unexpectedly
while	Starts a while loop

Example Program

```
#include <stdio.h>

int main()
{
    int num = 10;
    if (num > 0)
    {
        printf("Positive Number");
    }
    return 0;
}
```

Literals in C

A **literal** is a fixed value written directly in the source code. Unlike variables, the value of a literal **cannot be changed** during program execution.

Types of Literals

1. Integer Literals

Whole numbers without a decimal point.

Examples:

10

-25

0

100

2. Floating-Point Literals

Numbers with a decimal point or exponent.

Examples:

3.14

-2.5

0.001

6.02e23

3. Character Literals

A single character enclosed in single quotes.

Examples:

'A'

'9'

'#'

4. String Literals

A sequence of characters enclosed in double quotes.

Examples:

"Hello"

"C Programming"

"Welcome"

5. Escape Sequence Literals

Special character literals that begin with a backslash (\).

Escape Sequence	Meaning
\n	New line
\t	Horizontal tab
\\	Backslash
\"	Double quotation mark
\'	Single quotation mark

Example Program

```
#include <stdio.h>

int main()
{
    printf("Integer: %d\n", 100);
    printf("Float: %.2f\n", 3.14);
    printf("Character: %c\n", 'A');
    printf("String: %s\n", "Hello");
    return 0;
}
```

Output

```
Integer: 100
Float: 3.14
Character: A
String: Hello
```

1.13 Expressions, Single line and multiline comments

Expressions

In **C programming**, an **expression** is a combination of **variables, constants, operators, and function calls** that evaluates to a single value. An **expression** is a valid combination of operands and operators that produces a result.

Syntax

```
operand operator operand
```

Examples

10 + 20 // Arithmetic expression

a + b // Variable expression

x * y + 5 // Complex expression

a > b // Relational expression

x && y // Logical expression

Types of Expressions in C

1. Arithmetic Expression

- Uses arithmetic operators (+, -, *, /, %)
- Example:
- `int sum = a + b;`

2. Relational Expression

- Uses relational operators (>, <, >=, <=, ==, !=)
- Returns 1 (true) or 0 (false).
- Example:
- `a > b`

3. Logical Expression

- Uses logical operators (&&, ||, !)
- Example:
- `(a > b) && (b > c)`

4. Assignment Expression

- Uses the assignment operator (=)
- Example:
- `x = 50;`

5. Increment/Decrement Expression

- Uses ++ or --
- Example:
- `i++;`
- `--j;`

6. Conditional (Ternary) Expression

- Uses the ?: operator.
- Example:
- `max = (a > b) ? a : b;`

7. Bitwise Expression

- Uses bitwise operators (&, |, ^, ~, <<, >>)
- Example:
- `x & y`

Example Program

```
#include <stdio.h>

int main() {
    int a = 10, b = 5, c;
    c = a + b * 2; // Expression
    printf("Result = %d", c);
    return 0;
}
```

Output:

Result = 20

Single-line and Multi-line Comments in C

Comments are used to **explain the code** and improve readability. The C compiler ignores comments during compilation.

1. Single-line Comment

A **single-line comment** begins with // and continues until the end of the line.

Syntax:

```
// This is a single-line comment
```

Example:

```
#include <stdio.h>

int main() {
    // Declare a variable
    int num = 10;
    // Print the value
```

```

printf("%d", num);

return 0;

}

```

2. Multi-line Comment

A **multi-line comment** begins with `/*` and ends with `*/`. It can span multiple lines.

Syntax:

```

/* This is a

multi-line

comment */

```

Example:

```

#include <stdio.h>

int main() {

    /* This program

    prints a message

    on the screen */

    printf("Hello, World!");

    return 0;

}

```

1.14 Datatype classification : Basic data type, Derived Data type, User defined data type

Basic Data Types in C

A **data type** specifies the type of data a variable can store and the amount of memory allocated for it. **Basic data types** are the fundamental data types provided by the C language to store different kinds of values.

Basic Data Types

Data Type	Description	Typical Size*	Example
int	Stores whole numbers (integers)	4 bytes	int age = 20;
char	Stores a single character	1 byte	char grade = 'A';
float	Stores single-precision decimal numbers	4 bytes	float price = 99.5;

Data Type	Description	Typical Size*	Example
double	Stores double-precision decimal numbers	8 bytes	double pi = 3.1415926535;
void	Represents no value or no data type	—	void display();

*The exact size depends on the compiler and system architecture.

1. Integer (int)

- Stores whole numbers (positive, negative, or zero).
- **Example:**

```
int marks = 85;
```

2. Character (char)

- Stores a single character enclosed in single quotes.
- **Example:**

```
char grade = 'A';
```

3. Float (float)

- Stores decimal (floating-point) values with single precision.
- **Example:**

```
float temperature = 36.5;
```

4. Double (double)

- Stores decimal values with higher precision than float.
- **Example:**

```
double salary = 45678.987654;
```

5. Void (void)

- Represents the absence of a value.
- Commonly used for functions that do not return a value.
- **Example:**

```
void display()
{
    printf("Hello");
}
```

Example Program

```
#include <stdio.h>

int main() {
    int age = 20;
    char grade = 'A';
    float marks = 89.5;
    double pi = 3.1415926535;
    printf("Age = %d\n", age);
    printf("Grade = %c\n", grade);
    printf("Marks = %.1f\n", marks);
    printf("Pi = %lf\n", pi);
    return 0;
}
```

Output

```
Age = 20
Grade = A
Marks = 89.5
Pi = 3.141593
```

Derived Data Types in C

A **derived data type** is a data type that is **created from the basic (primitive) data types**. Derived data types allow programmers to store and manipulate collections of data or memory addresses. **Derived data types** are data types formed from basic data types using C language constructs such as arrays, pointers, functions, and structures.

Types of Derived Data Types

Derived Data Type	Description	Example
Array	Stores multiple values of the same data type in contiguous memory locations.	int marks[5];
Pointer	Stores the memory address of another variable.	int *ptr;
Function	A block of code that performs a specific task and may return a value.	int add(int a, int b);

Derived Data Type	Description	Example
Structure	Groups variables of different data types under a single name.	struct Student { int id; char name[20]; };

1. Array

- Stores multiple elements of the same data type.
- Elements are accessed using an index.

Example:

```
int numbers[5] = {10, 20, 30, 40, 50};
```

2. Pointer

- A pointer stores the address of a variable.
- Declared using the * symbol.

Example:

```
int a = 10;
int *ptr = &a;
```

3. Function

- A function is a reusable block of code that performs a specific task.
- It can accept parameters and return a value.

Example:

```
int add(int x, int y)
{
    return x + y;
}
```

4. Structure

- A structure combines variables of different data types into a single user-defined type.

Example:

```
struct Student
{
    int id;
    char name[20];
};
```

Example Program

```
#include <stdio.h>

struct Student
{
    int id;
    char grade;
};

int main()
{
    int numbers[3] = {10, 20, 30};
    int a = 50;
    int *ptr = &a;
    struct Student s1 = {101, 'A'};
    printf("First Array Element: %d\n", numbers[0]);
    printf("Pointer Value: %d\n", *ptr);
    printf("Student ID: %d\n", s1.id);
    return 0;
}
```

Output

First Array Element: 10

Pointer Value: 50

Student ID: 101

User-Defined Data Types in C

A **user-defined data type** is a data type **created by the programmer** to organize and manage data more efficiently. These data types are built using the features provided by the C language. **User-defined data types** are data types that allow programmers to create their own data types according to the requirements of a program.

Types of User-Defined Data Types

Data Type	Description	Example
Structure (struct)	Groups variables of different data types under one name.	struct Student

Data Type	Description	Example
Union (union)	Stores different data types in the same memory location.	union Data
Enumeration (enum)	Defines a set of named integer constants.	enum Day
Typedef (typedef)	Creates a new name (alias) for an existing data type.	typedef int Integer;

1. Structure (struct)

- Combines variables of different data types into a single unit.
- Each member has its own memory.

Example:

```
struct Student
{
    int id;
    char name[20];
};
```

2. Union (union)

- Similar to a structure, but all members share the same memory location.
- Only one member can hold a valid value at a time.

Example:

```
union Data
{
    int i;
    float f;
    char c;
};
```

3. Enumeration (enum)

- Defines a list of named integer constants.
- Improves code readability.

Example:

```
enum Day
```

```
{  
    MON,  
    TUE,  
    WED,  
    THU,  
    FRI  
};
```

4. Typedef (typedef)

- Creates an alias (new name) for an existing data type.
- Makes code easier to read.

Example:

```
typedef int Integer;
```

```
Integer age = 20;
```

Example Program

```
#include <stdio.h>
```

```
typedef int Integer;
```

```
struct Student
```

```
{  
    int id;  
    char grade;
```

```
};
```

```
enum Day
```

```
{  
    MON,  
    TUE,  
    WED  
};
```

```

int main()
{
    Integer age = 20;
    struct Student s1 = {101, 'A'};
    enum Day today = MON;
    printf("Age = %d\n", age);
    printf("Student ID = %d\n", s1.id);
    printf("Day = %d\n", today);

    return 0;
}

```

Output

```

Age = 20
Student ID = 101
Day = 0

```

Difference Between Basic, Derived, and User-Defined Data Types

Basic Data Types	Derived Data Types	User-Defined Data Types
Predefined by C.	Created from basic data types.	Created by the programmer.
Store single values.	Store collections, addresses, or functions.	Define custom data structures and aliases.
Examples: int, char, float, double.	Examples: Array, Pointer, Function.	Examples: struct, union, enum, typedef.

DLD — CT1 Crash Revision

Number Systems • Codes • Boolean Algebra • Logic Gates

Target: 40+/50. Priority order for tonight — do sections in this order if short on time: 1) Conversions & Complements, 2) Codes, 3) Boolean Laws + De Morgan's, 4) Gates + Truth Tables, 5) Universal Gates, 6) Simplification, 7) Definitions.

1. NUMBER SYSTEM CONVERSIONS [HIGH PRIORITY]

Base values: Binary=2, Octal=8, Decimal=10, Hexadecimal=16

Decimal → Binary/Octal/Hex

Repeatedly divide by the base, note remainders bottom-to-top.

Example: $45 \rightarrow \text{Binary} \rightarrow 45 \div 2 = 22 \text{ r}1, 22 \div 2 = 11 \text{ r}0, 11 \div 2 = 5 \text{ r}1, 5 \div 2 = 2 \text{ r}1, 2 \div 2 = 1 \text{ r}0, 1 \div 2 = 0 \text{ r}1 \rightarrow \text{read bottom-up} = \mathbf{101101}$

Binary/Octal/Hex → Decimal

Multiply each digit by (base)^{position}, position counted from 0 at rightmost.

Example: $1011 \text{ (binary)} = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = \mathbf{11}$

Example: $3A \text{ (hex)} = 3 \times 16^1 + 10 \times 16^0 = 48 + 10 = \mathbf{58}$

Binary ↔ Octal (group in 3s)

Binary→Octal: group bits in 3s from right, pad zeros on left if needed, convert each group.

Example: $101110 \rightarrow 101 \ 110 \rightarrow 5 \ 6 \rightarrow \mathbf{56 \text{ (octal)}}$

Octal→Binary: convert each octal digit to 3-bit binary and join.

Example: $47 \text{ (octal)} \rightarrow 4=100, 7=111 \rightarrow \mathbf{100111}$

Binary ↔ Hexadecimal (group in 4s)

Binary→Hex: group bits in 4s from right, pad zeros if needed, convert each group.

Example: $10110111 \rightarrow 1011 \ 0111 \rightarrow B \ 7 \rightarrow \mathbf{B7 \text{ (hex)}}$

Hex→Binary: convert each hex digit to 4-bit binary and join.

Example: $2F \text{ (hex)} \rightarrow 2=0010, F=1111 \rightarrow \mathbf{00101111}$

Octal ↔ Hexadecimal

No direct shortcut — convert Octal→Binary→Hex (or Hex→Binary→Octal) via the 3-bit/4-bit grouping above.

Hex digit reference: A=10, B=11, C=12, D=13, E=14, F=15

2. BINARY ARITHMETIC [HIGH PRIORITY]

Addition rules

$0+0=0$, $0+1=1$, $1+0=1$, $1+1=10$ (0 carry 1), $1+1+1=11$ (1 carry 1)

Example: $1011 + 1101$

```
1011
+ 1101
-----
11000
```

Subtraction rules

$0-0=0$, $1-0=1$, $1-1=0$, $0-1=1$ (borrow 1 from next column)

Example: $1101 - 1011 = 0010$

Tip: exam-friendly method is to add the 2's complement instead of borrowing directly (see Section 3).

Multiplication rules

Same as decimal long multiplication but with $0 \times 0 = 0$, $0 \times 1 = 0$, $1 \times 0 = 0$, $1 \times 1 = 1$, then add shifted partial products.

Example: 101×11

```
101
× 011
-----
101
101_
-----
1111
```

3. COMPLEMENTS [HIGH PRIORITY]

1's Complement (Binary)

Flip every bit: $0 \rightarrow 1$ and $1 \rightarrow 0$.

Example: $1010 \rightarrow 1\text{'s complement} = 0101$

2's Complement (Binary)

Take 1's complement, then add 1.

Example: $1010 \rightarrow 1\text{'s comp} = 0101 \rightarrow +1 = 0110$

Shortcut: from the right, keep digits unchanged until the first '1', then flip all bits after it.

9's Complement (Decimal)

Subtract each digit from 9.

Example: $4523 \rightarrow 9999 - 4523 = 5476$

10's Complement (Decimal)

Take 9's complement, then add 1.

Example: $4523 \rightarrow 9\text{'s comp} = 5476 \rightarrow +1 = 5477$

Why complements matter: used to perform subtraction via addition in digital circuits (most likely to appear as a 'why is 2's complement used' theory question).

4. CODES — WEIGHTED & NON-WEIGHTED [HIGH PRIORITY]

Weighted codes: each bit position has a fixed weight (value). Non-weighted codes: positions have no fixed weight — used for error detection or special sequencing.

Code	Type	Weights / Rule
8421 BCD	Weighted	Standard binary weights 8,4,2,1 — same as 4-bit binary per decimal digit
2421	Weighted	Weights 2,4,2,1 — self-complementing code
5211	Weighted	Weights 5,2,1,1
Excess-3	Non-weighted	Add 3 (0011) to the 8421 BCD code of the digit
Gray Code	Non-weighted	Only one bit changes between successive values

BCD (8421) table for 0–9

Decimal	0	1	2	3	4	5	6	7	8	9
8421 BCD	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

Excess-3 table for 0–9 (BCD + 0011)

Decimal	0	1	2	3	4	5	6	7	8	9
Excess-3	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100

5. CODE CONVERSIONS [HIGH PRIORITY]

Binary → Gray Code

MSB of Gray = MSB of Binary. Each next Gray bit = XOR of current and previous Binary bit.

Example: Binary 1011 → Gray: 1, $1 \oplus 0 = 1$, $0 \oplus 1 = 1$, $1 \oplus 1 = 0$ → **1110**

Gray Code → Binary

MSB of Binary = MSB of Gray. Each next Binary bit = XOR of previous Binary bit and current Gray bit.

Example: Gray 1110 → Binary: 1, $1 \oplus 1 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$ → **1011**

Decimal → BCD / BCD → Decimal

Convert each decimal digit separately to its 4-bit 8421 code (and vice versa — split into groups of 4 bits, convert each group back to decimal digit).

Example: 29 → BCD = 0010 1001

Decimal → Excess-3 / Excess-3 → Decimal

Decimal→Excess-3: get BCD of each digit, add 0011 (3) to each 4-bit group.

Excess-3→Decimal: subtract 0011 (3) from each 4-bit group, then read as BCD.

Example: 8 → BCD 1000 → +0011 = **1011 (Excess-3)**

6-9. BOOLEAN ALGEBRA [HIGH PRIORITY]

Basics

- **Boolean variable:** a symbol (A, B, C...) representing a value of 0 or 1.
- **Boolean expression:** a combination of variables and operators, e.g. $A \cdot B + C'$.
- **Boolean operators:** AND ($\cdot$), OR ($+$), NOT ($'$ or bar) — the three fundamental operations.

Boolean Algebra Laws

Law	AND form	OR form
Commutative	$A \cdot B = B \cdot A$	$A + B = B + A$
Associative	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	$(A + B) + C = A + (B + C)$
Distributive	$A \cdot (B + C) = A \cdot B + A \cdot C$	$A + (B \cdot C) = (A + B) \cdot (A + C)$
Identity	$A \cdot 1 = A$	$A + 0 = A$
Null (Domination)	$A \cdot 0 = 0$	$A + 1 = 1$
Idempotent	$A \cdot A = A$	$A + A = A$
Complement	$A \cdot A' = 0$	$A + A' = 1$
Absorption	$A \cdot (A + B) = A$	$A + (A \cdot B) = A$
Involution	$(A')' = A$	—

Duality Property

To find the dual of an expression: swap AND $\leftrightarrow$ OR, and swap 0 $\leftrightarrow$ 1. Variables stay unchanged.

Example: $A + (B \cdot C) \rightarrow \text{dual} \rightarrow A \cdot (B + C)$

Example: $A \cdot 1 = A \rightarrow \text{dual} \rightarrow A + 0 = A$

De Morgan's Theorems

First theorem: $(A \cdot B)' = A' + B'$ (NAND of inputs = OR of complemented inputs)

Second theorem: $(A + B)' = A' \cdot B'$ (NOR of inputs = AND of complemented inputs)

Proof approach: build a truth table for LHS and RHS separately for all combinations of A,B — if columns match exactly, the theorem is proved.

A	B	A'	B'	$(A \cdot B)'$	$A' + B'$	$(A + B)'$	$A' \cdot B'$
0	0	1	1	1	1	1	1
0	1	1	0	1	1	0	0
1	0	0	1	1	1	0	0
1	1	0	0	0	0	0	0

$(A \cdot B)'$ matches $A' + B'$ in every row $\rightarrow$ 1st theorem proved. $(A + B)'$ matches $A' \cdot B'$ $\rightarrow$ 2nd theorem proved.

Application: De Morgan's laws are used to convert AND/OR circuits into NAND/NOR-only circuits, and to simplify complemented expressions before implementation.

10-13. LOGIC GATES [HIGH PRIORITY]

Basic Gates: AND, OR, NOT

Gate	Expression	Truth Table (Inputs→Output)	Real-life use
AND	$Y = A \cdot B$	00→0, 01→0, 10→0, 11→1	Series switch circuit — both conditions must be true
OR	$Y = A + B$	00→0, 01→1, 10→1, 11→1	Parallel switch circuit — either condition triggers output
NOT	$Y = A'$	0→1, 1→0	Inverter — used to complement a signal

Derived Gates: NAND, NOR, XOR, XNOR

Gate	Expression	Truth Table (Inputs→Output)	Real-life use
NAND	$Y = (A \cdot B)'$	00→1, 01→1, 10→1, 11→0	Universal gate — used to build all other gates
NOR	$Y = (A + B)'$	00→1, 01→0, 10→0, 11→0	Universal gate — used in memory (SR latch)
XOR	$Y = A \oplus B$	00→0, 01→1, 10→1, 11→0	Adders, comparators, parity/error-detection circuits
XNOR	$Y = (A \oplus B)'$	00→1, 01→0, 10→0, 11→1	Equality checkers/comparators

Symbols: draw AND as a 'D' shape, OR as a curved-back shield shape, NOT as a triangle with a bubble at the tip. NAND/NOR/XOR/XNOR = AND/OR/XOR shape + a bubble at output (XOR/XNOR also have a curved double line at input).

Basic vs Derived Gates

- **Basic gates:** AND, OR, NOT — the fundamental building blocks, cannot be broken down further.
- **Derived gates:** NAND, NOR, XOR, XNOR — built by combining basic gates.

Universal Gates: NAND & NOR

A gate is 'universal' if ANY logic gate (AND, OR, NOT) can be built using only that gate.

Target Gate	Using only NAND	Using only NOR
NOT (A')	Tie both NAND inputs to A	Tie both NOR inputs to A
AND ($A \cdot B$)	NAND(A,B) then NOT that output	NOR(NOT A, NOT B)
OR ($A + B$)	NOT A, NOT B, then NAND them	NOR(A,B) then NOT that output

Memory trick: NOT = self-fed gate. AND = NAND+NOT. OR = NOT the inputs first, then NAND (for NAND-only OR) — this is essentially De Morgan's theorem in circuit form.

14-16. CIRCUITS, TRUTH TABLES & SIMPLIFICATION [PRIORITY]

Logic diagram $\leftrightarrow$ Boolean expression

- To draw a circuit from an expression: work from innermost brackets outward, one gate per operation.
- To write an expression from a circuit: trace signal flow from inputs to final output, naming each gate's operation.
- Always match the number of inputs on each gate to the number of terms feeding it.

Truth Table Construction

- For n variables, there are 2^n rows — list all combinations in binary order (00...0 to 11...1).
- Build intermediate columns for sub-expressions before the final output column — reduces mistakes.
- To verify an equivalence, build truth tables for both sides and check every row matches (as done for De Morgan's above).

Boolean Expression Simplification — worked example

Simplify: $Y = A \cdot B + A \cdot B' + A' \cdot B$

$$Y = A(B+B') + A'B \quad [\text{Distributive}]$$

$$Y = A(1) + A'B \quad [\text{Complement law: } B+B'=1]$$

$$Y = A + A'B \quad [\text{Identity law}]$$

$$Y = A + B \quad [\text{Absorption-type identity: } A+A'B = A+B]$$

Simplify using De Morgan's: $Y = (A'+B)'$

$$Y = (A')' \cdot (B')' = A \cdot B \quad [\text{Apply De Morgan's, then Involution law}]$$

17. QUICK-FIRE DEFINITIONS [PRIORITY]

Truth table: A table listing all possible input combinations of a logic circuit and their corresponding outputs.

Derived gates: Logic gates built by combining basic gates (AND, OR, NOT) — e.g. NAND, NOR, XOR, XNOR.

Weighted code: A code where each bit position carries a fixed numerical weight, e.g. 8421 BCD.

Non-weighted code: A code where bit positions carry no fixed weight, e.g. Gray code, Excess-3.

Duality property: Every Boolean identity remains valid if $AND \leftrightarrow OR$ and $0 \leftrightarrow 1$ are interchanged throughout.

Universal gate: A gate (NAND or NOR) that can be used alone to implement any other logic gate/function.

Boolean algebra: A mathematical system dealing with binary variables and logical operations (AND, OR, NOT), used to analyze/simplify digital circuits.

Logic gate: An electronic circuit that performs a basic Boolean operation on one or more binary inputs to produce a single binary output.

Final tip: In DLD papers, conversions/complements/codes are usually fast, guaranteed marks if you drill 5-10 practice problems tonight. Boolean laws + De Morgan's + gates are usually a mix of 'state and prove' and 'simplify this expression' — memorize the law table and the worked example pattern above and you can adapt it to any similar expression in the exam.

